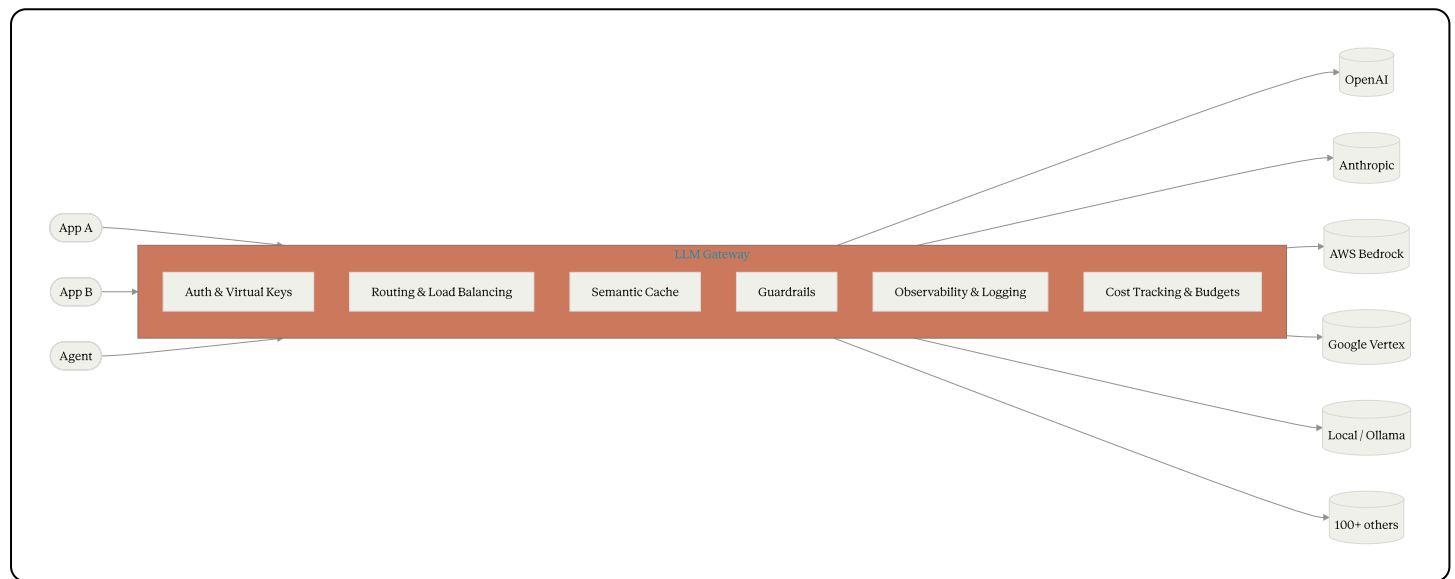


LLM Gateways: The Complete 2026 Guide

Portkey, Bifrost, LiteLLM, and the full landscape — what they are, how they work, and which one to choose for your production AI stack.

1. What Is an LLM Gateway?

An **LLM Gateway** is a centralized proxy and control layer that sits **between your application and one or more LLM providers**. It provides a unified interface to route requests, manage credentials, enforce policies, observe model behavior, and optimize cost and reliability — across any combination of providers and models.



Why LLM Gateways have become critical infrastructure in 2026

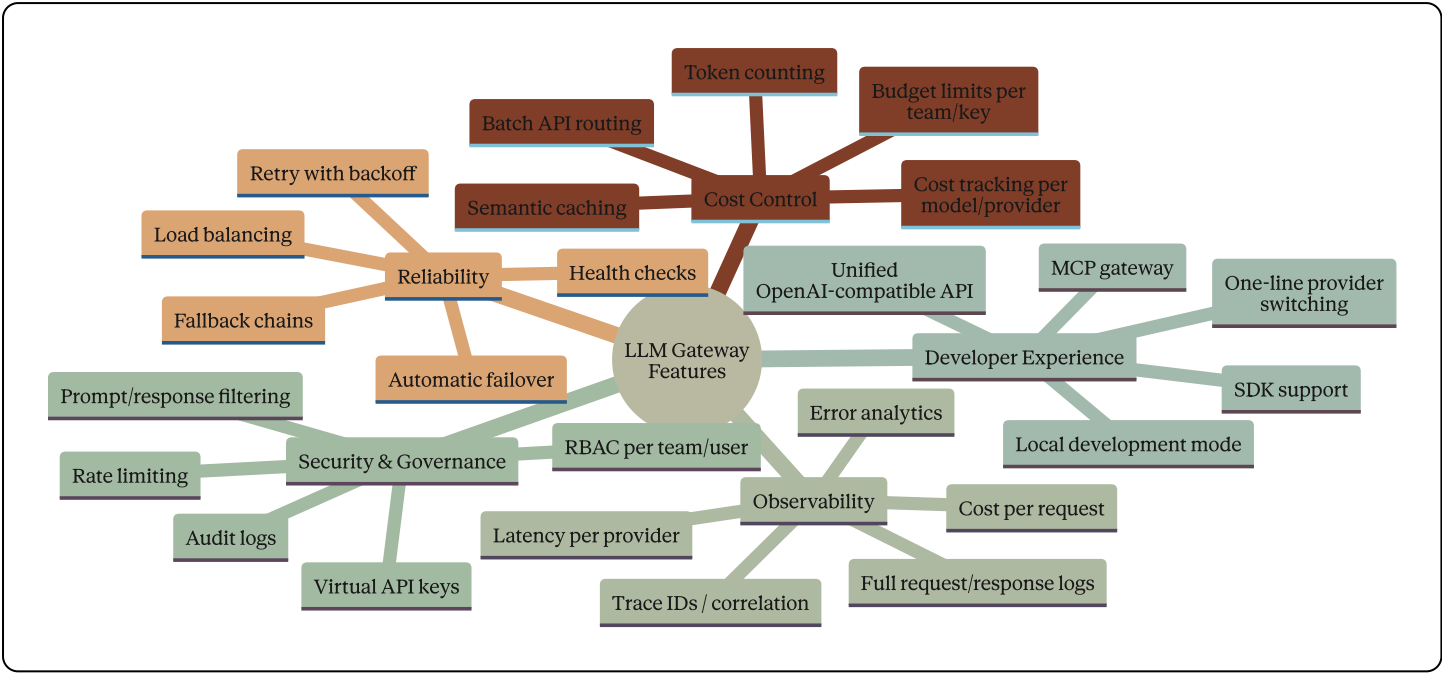
The LLM provider landscape is increasingly fragmented — every organization runs multiple models from multiple providers. Without a gateway:

- Each team manages its own API keys (security nightmare)
- No unified cost visibility across providers and teams
- A provider outage takes down your application
- Switching models requires rewriting application code
- No centralized rate limiting or budget controls
- Compliance teams can't audit what prompts and responses flow through the system
- MCP (Model Context Protocol) agent tool connections have no governance layer

Gartner named LLM Gateways an "emerging must-have" for enterprise AI in 2025. The category has seen explosive adoption: Portkey reports 1 trillion tokens/day, 24,000+ organizations, and \$180M in annualized AI spend governed through its gateway as of March 2026.

2. The Core Feature Set

Every serious LLM gateway covers these five pillars:



Feature-by-feature breakdown

Feature	What it does	Why it matters
Unified API	One endpoint, one format (OpenAI-compatible), all providers	Switch from GPT-5 to Claude in 1 line of code
Automatic failover	Route to backup provider on timeout/error	99.99% uptime even during provider outages
Load balancing	Distribute load across keys/models/providers by weight or latency	Prevent rate limit errors at scale
Semantic caching	Match semantically similar prompts to cached responses	30-70% cost reduction on repeated or similar queries
Virtual keys	Proxy keys that map to real provider keys stored in vault	Never expose raw API keys; revoke/rotate without code changes
Budget controls	Per-team, per-project, per-user spend limits	Finance teams can enforce AI budgets centrally
Rate limiting	Per-key or per-user request limits	Prevent runaway costs from a single bad deployment
Observability	Full logs, traces, latency breakdowns	Debug issues, audit compliance, optimize performance

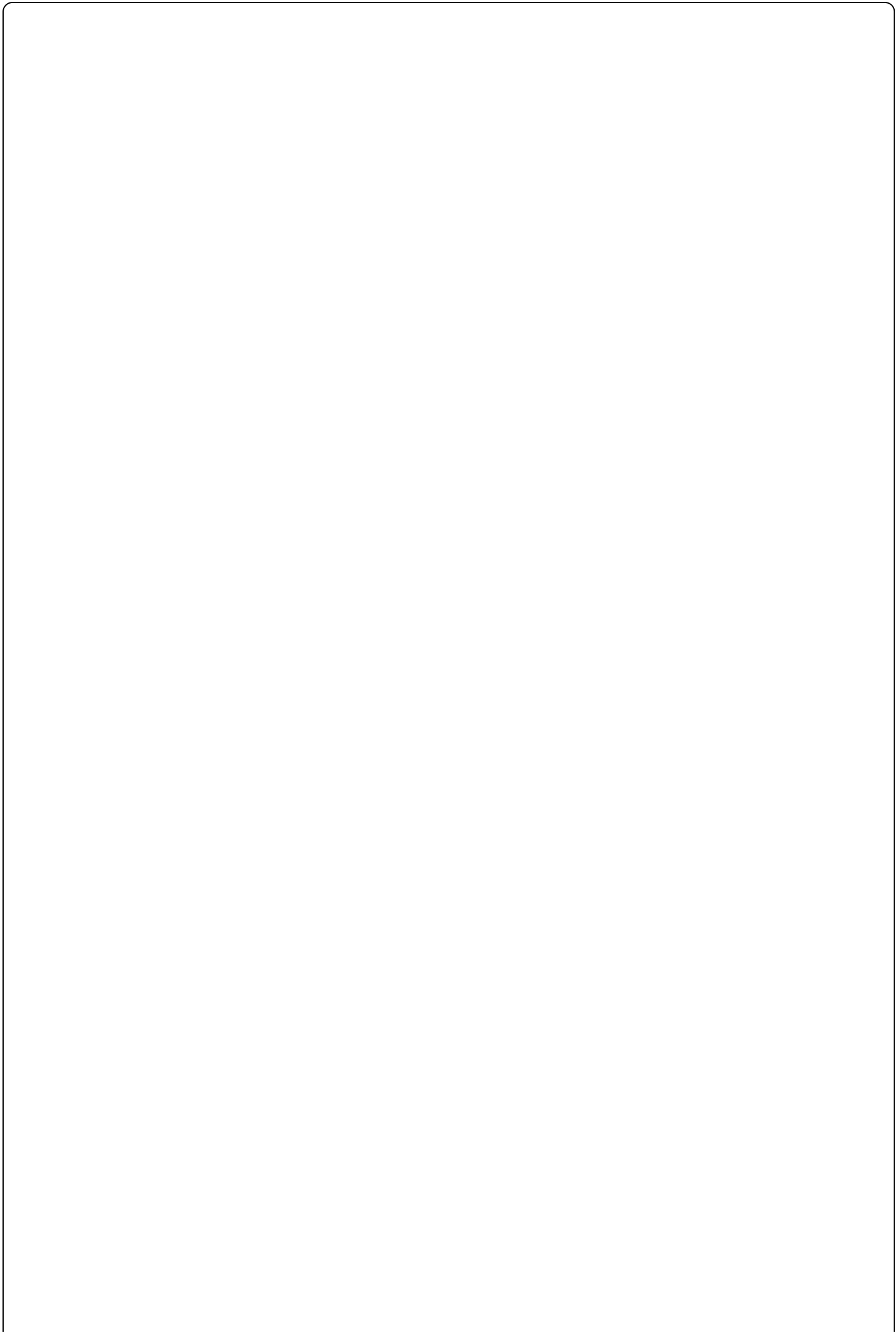
Feature	What it does	Why it matters
Guardrails	Input/output filtering (varies by gateway)	Safety and policy enforcement at the infrastructure level
Prompt management	Version-controlled prompt templates	Consistent prompts across teams; A/B testing
MCP gateway	Centralized control for Model Context Protocol tool servers	Agent governance — who can use which tools, with audit logs

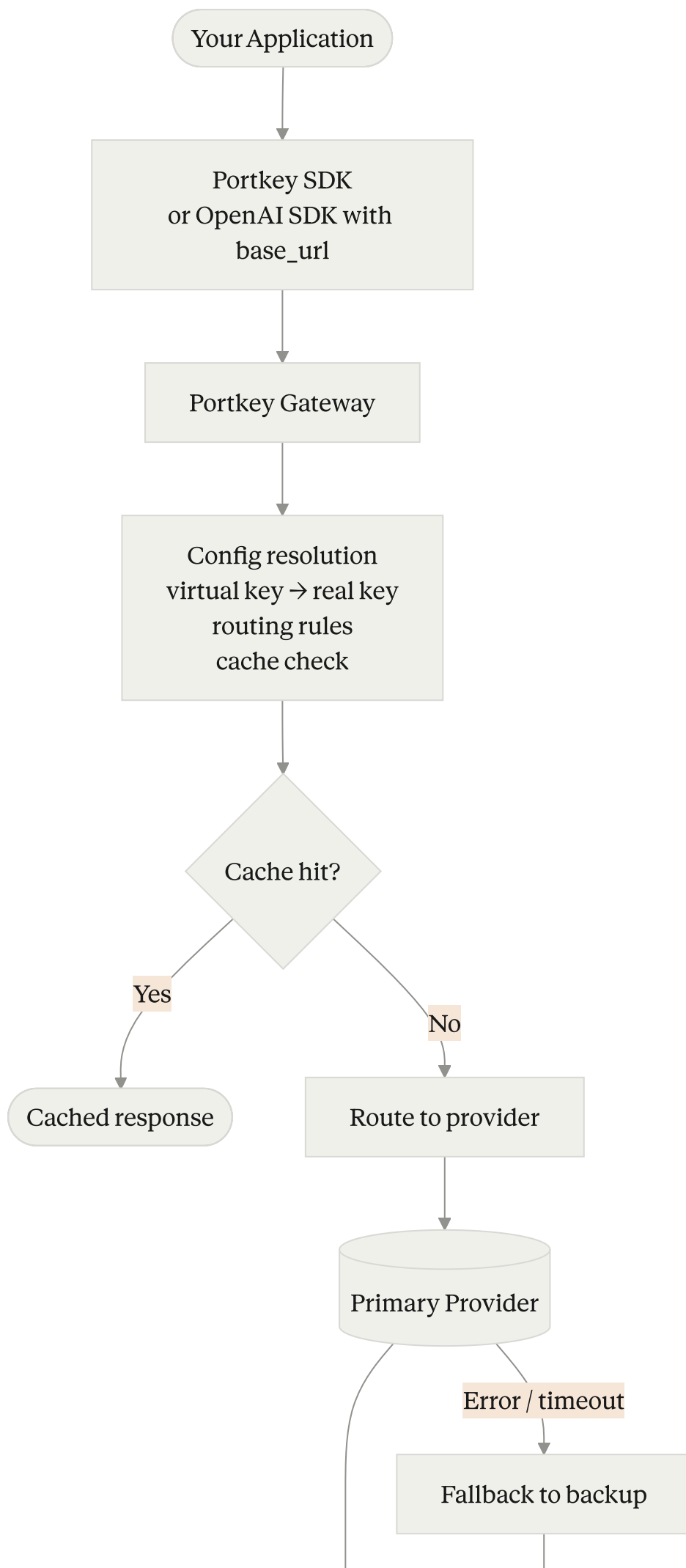
3. The Gateways, In Depth

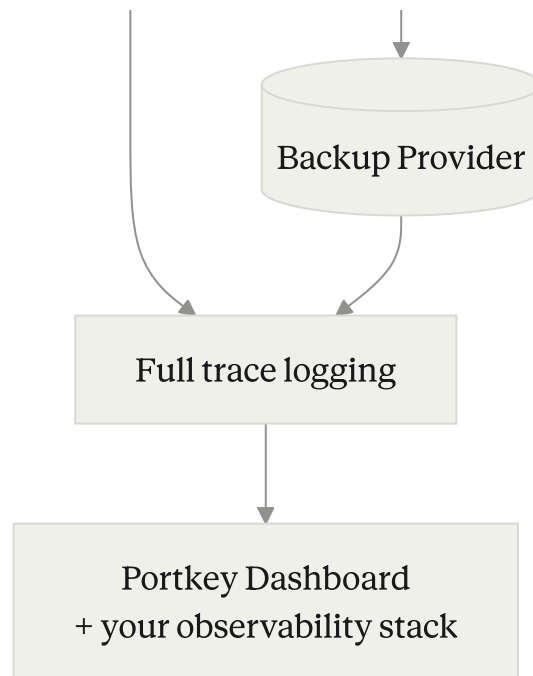
3.1 Portkey — "The Control Plane for Production AI"

What it is: The most feature-complete LLM gateway platform, positioned as a full AI control plane covering gateway, observability, guardrails, governance, and prompt management. Went fully open-source in March 2026.

How it works: Portkey sits as a transparent proxy in front of all LLM calls. The `portkey-ai` SDK is a drop-in replacement for the OpenAI SDK — changing `base_url` routes all calls through Portkey's gateway, which applies the configured routing config, caching, logging, and guardrails before forwarding to the provider.







Key features (2026):

- **1,600+ LLM support:** OpenAI, Anthropic, Azure, Bedrock, Vertex, Mistral, Cohere, Groq, Together, Fireworks, Ollama, Hugging Face, 1,600+ total. Also vision, audio, image generation.
- **OpenAI-compatible API:** drop-in replacement with 1-line config change
- **Semantic caching:** fuzzy matching on embedding similarity — best-in-class caching among all gateways
- **Intelligent routing:** static weights, latency-based, cost-based, conditional (route to model X if context > N tokens), canary testing (% traffic to new model)
- **Automatic retries:** up to 5 retries with exponential backoff per request
- **Fallback chains:** specify fallback providers/models with trigger conditions (errors, timeouts, specific status codes)
- **Virtual keys + Key Vault:** store real provider keys encrypted; create virtual keys with independent budgets, rate limits, and access controls. Rotate/revoke without code changes.
- **RBAC + org management:** multi-team hierarchy with role-based access control
- **Built-in guardrails:** 50+ guardrails covering prompt injection, PII, toxicity, regex, JSON validation, custom. Plus integrations with NeMo Guardrails, Lakera, Pillar Security.
- **Full observability:** per-request traces showing user, model tried, failure reason, fallback used, latency per step, cost. Export to Datadog, Grafana, S3, etc.
- **Prompt management:** versioned prompt templates with A/B testing
- **MCP Gateway:** centralized control plane for MCP servers — single auth layer, access control (who can use which tools), full audit log of every tool call, identity forwarding. Works with Claude Desktop, Cursor, VS Code.
- **Batch API routing:** cloud provider batch APIs for async workloads
- **Budget alerts:** real-time cost monitoring with threshold alerts

Architecture philosophy: Everything through the gateway = total observability. Full trace shows: which user made the call → which models were tried → why they failed → which fallback was used → exact latency per step → exact cost.

2026 milestones:

- Went fully open-source (March 2026), including governance, observability, authentication, cost controls
- \$15M Series A (Feb 2026), backed by Elevation Capital and Lightspeed
- 1T+ tokens/day, 120M+ AI requests/day, \$180M+ annualized AI spend governed, 24,000+ organizations
- MCP Gateway launched as fastest-adopted product in company history

Strengths: Most complete feature set, best-in-class observability depth, semantic caching, 1,600+ providers, strong enterprise story (SOC 2, HIPAA), MCP gateway, fully OSS now.

Weaknesses: More surface area to learn; Python-based core (though JS/TS SDKs available); HIPAA BAA process takes 2-3 weeks; managed cloud version means prompts leave your infrastructure (self-host to avoid).

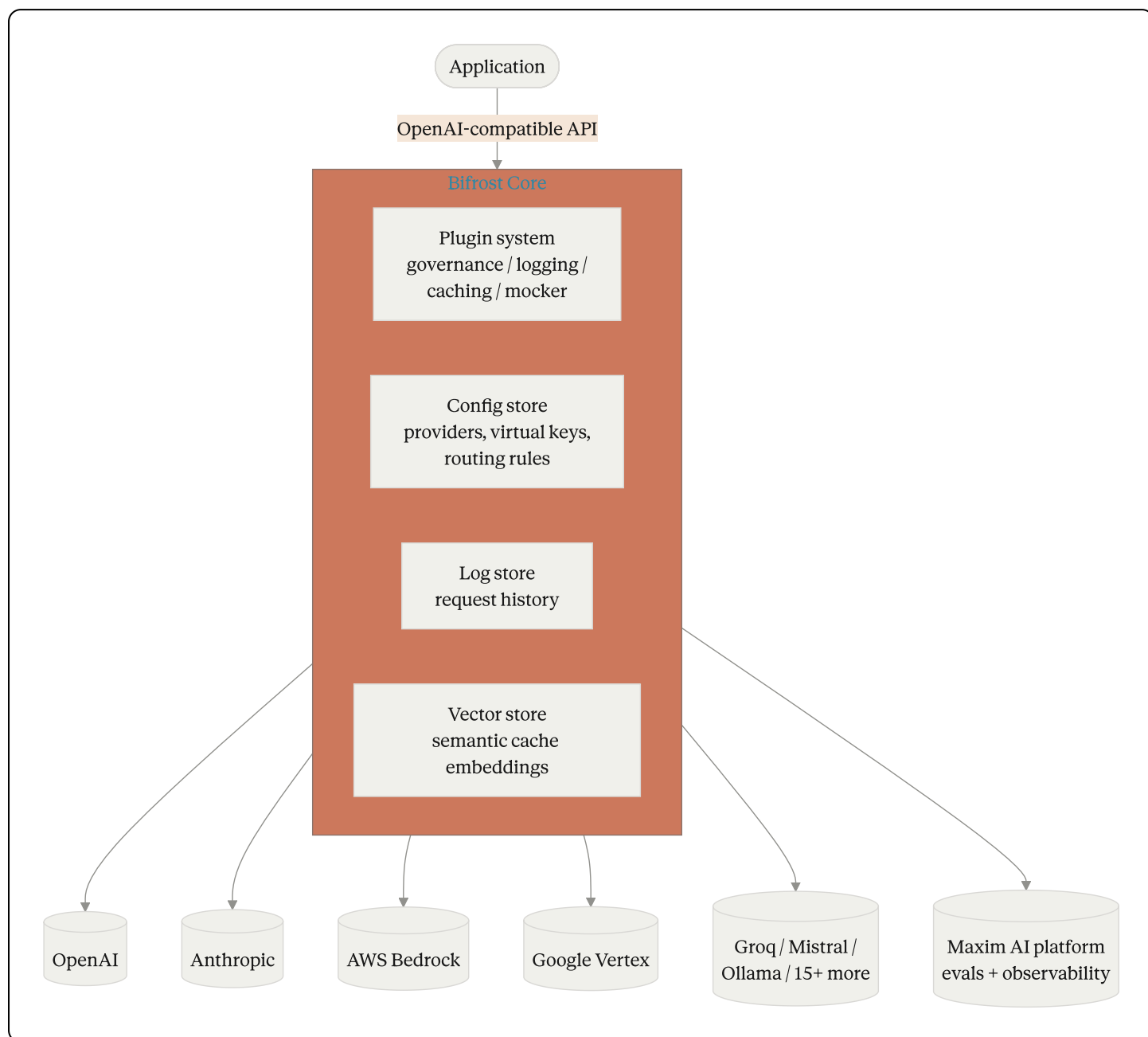
Best for: Teams needing a **full AI control plane** — routing + observability + guardrails + prompt management + governance + MCP governance — with enterprise-grade reliability and auditability. Especially strong for multi-team enterprise deployments.

License	Apache 2.0 (fully open-sourced March 2026)
GitHub	github.com/Portkey-AI/gateway
Stars	~7k+
Language	Python, TypeScript
Deployment	SaaS (Portkey Cloud) or self-host (Docker/K8s)
Install	<code>pip install portkey-ai</code>
Docs	portkey.ai/docs

3.2 Bifrost (Maxim AI) — "Fastest Enterprise Gateway"

What it is: The **highest-performance open-source LLM gateway** — written in Go, with 11µs overhead at 5,000 RPS, making it approximately 50× faster than LiteLLM in sustained benchmarks. Built by Maxim AI, Apache 2.0.

How it works: Bifrost is a Go binary (or Docker container) that proxies LLM requests. Its Go architecture avoids Python's GIL bottleneck, enabling true parallelism under high concurrency. Configuration is file-based or via Web UI; the API is fully OpenAI-compatible, requiring only a `base_url` change.



Key features:

- **<11μs overhead at 5,000 RPS** — the fastest gateway by published benchmarks
- **Zero-config startup:** `npx -y @maximhq/bifrost` or `docker run maximhq/bifrost` — running in 30 seconds
- **OpenAI-compatible API:** drop-in with 1-line change; supports OpenAI, Anthropic, Google GenAI, LangChain, LiteLLM compatibility modes
- **15+ providers:** OpenAI, Anthropic, AWS Bedrock, Google Vertex, Azure, Cohere, Mistral, Ollama, Groq, Cerebras, and more
- **Custom model support:** any deployed endpoint, fine-tuned models
- **Automatic failover:** provider-level and model-level fallback chains
- **Adaptive load balancing:** round-robin, weighted, latency-based, adaptive strategies

- **Semantic caching:** embedding-based fuzzy matching (backed by vector store plugin)
- **Virtual keys:** per-use-case keys with independent budgets and access control
- **Governance plugin:** budget limits per team/project/model, rate limiting, spending alerts
- **Plugin architecture:** modular system — governance, logging, caching, mocking, observability integrations
- **Web UI:** built-in real-time monitoring dashboard (no external setup needed)
- **OpenTelemetry support:** out-of-the-box OTel export for tracing
- **Cluster mode:** enterprise distributed deployment
- **Maxim AI integration:** deep integration with Maxim's eval + simulation + observability platform
- **MCP Gateway:** centralized MCP tool governance (newer, in active development)

Performance benchmarks (Bifrost team, March 2026 — verify with own workloads):

Gateway	Overhead at 5k RPS	Language	GIL-limited?
Bifrost	11µs	Go	No
LiteLLM	~8ms (P95 at 1k RPS)	Python	Yes
Portkey (self-host)	~1ms	Python/TS	Depends
Envoy AI Gateway	1-3ms	C++	No
Kong AI Gateway	~2ms	Lua/C	No

Strengths: Raw throughput leader, zero-config startup, Go runtime (no GIL ceiling), semantic caching, strong governance, Apache 2.0, native Maxim AI integration for evals.

Weaknesses: Fewer providers than LiteLLM (15+ vs 100+); smaller community; observability dashboard is basic compared to Portkey (deep analytics require Maxim platform); newer product, some enterprise features still developing; provider coverage gap may matter for teams needing exotic providers.

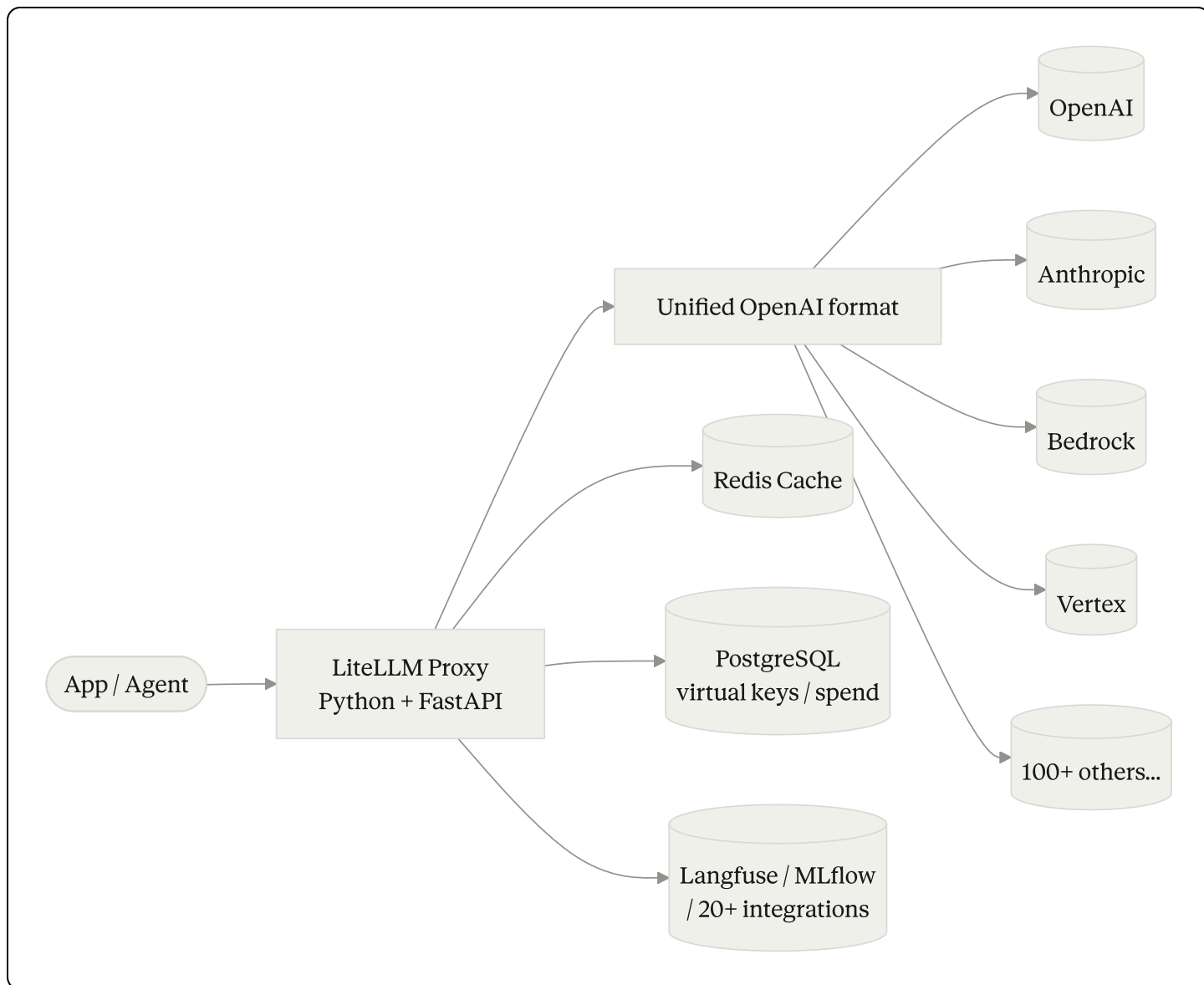
Best for: Teams where **performance is the primary constraint** — 5,000+ RPS, customer-facing real-time features, latency-sensitive applications. Also strong for teams wanting to pair their gateway with Maxim AI's evaluation platform in a single-vendor stack.

License	Apache 2.0
GitHub	github.com/maximhq/bifrost
Language	Go
Deployment	Self-host (npx / Docker / K8s)
Install	<code>npx -y @maximhq/bifrost</code> or <code>docker run maximhq/bifrost</code>
Docs	docs.getmaxim.ai/bifrost

3.3 LiteLLM — "100+ Providers, Maximum Ecosystem"

What it is: The most widely adopted open-source LLM proxy — MIT-licensed, Python-based, available as both an SDK and a proxy server. Known for the widest provider coverage (100+) and the largest community.

How it works: LiteLLM normalizes calls across all providers to OpenAI format. As an SDK, it wraps your code; as a proxy server, it sits in front of all LLM traffic. Uses a Redis-backed cache, virtual key management in PostgreSQL, and integrates with Langfuse, MLflow, and other observability platforms.



Key features:

- **100+ provider integrations** — the most comprehensive provider catalog: every major and minor cloud provider, open-source inference (Ollama, vLLM, LocalAI), and specialized providers
- **Unified OpenAI output format** across all providers (standardizes response schema)
- **MIT licensed** — most permissive open-source license
- **Python SDK:** `litellm.completion(model="claude-opus-4-6", ...)` — model string switches provider
- **Proxy server:** FastAPI-based, deployable via Docker
- **Retry and fallback logic** across multiple model deployments
- **Budget management:** per-project, per-team, per-API-key spend tracking (PostgreSQL)
- **Virtual keys** with team management
- **Latency-based routing:** `lowest_latency` router strategy
- **Cost-based routing:** `cheapest` router strategy
- **Usage-based routing:** `usage-based-routing` strategy
- **Redis caching:** exact-match response caching
- **MCP Gateway:** control tool access by team and key with granular permissions

- **Built-in guardrails:** keyword blocking, pattern detection, custom regex
- **20+ observability integrations:** Langfuse, MLflow, Helicone, Datadog, Prometheus, etc.
- **Helm chart** for Kubernetes deployment

The Python ceiling: LiteLLM's primary architectural limitation is Python's GIL. In sustained benchmarks, P95 latency at 1,000 RPS is ~8ms (gateway overhead) vs Bifrost's 11µs. At 2,000+ RPS, Python-based gateways start seeing memory spikes (8GB+) and cascading timeouts. Production teams at scale run multiple LiteLLM instances behind a load balancer — more infrastructure to manage.

Strengths: Largest provider catalog (100+), largest OSS community, MIT license, fastest iteration on new providers, well-documented, mature virtual key management.

Weaknesses: Python GIL limits throughput ceiling; no semantic caching (exact-match Redis only); MCP support is newer and less mature; observability requires external tools; governance features are basic (some behind commercial license).

Best for: Teams that need **maximum provider coverage** and are in a Python environment with moderate traffic (<500 RPS comfortable, workable to ~2k RPS with tuning). Excellent for rapid prototyping and research teams. The default choice when provider breadth matters most.

License	MIT
GitHub	github.com/BerriAI/litellm
Stars	~20k+
Language	Python
Deployment	SDK, Docker proxy, Helm
Install	<code>pip install litellm</code>
Docs	docs.litellm.ai

3.4 Other Notable Gateways

OpenRouter

What it is: Managed multi-model marketplace — sign up, get an API key, access 300+ models instantly. No infrastructure to deploy.

How it works: SaaS-only; you call `api.openrouter.ai` with any model name. OpenRouter handles routing, fallback, and billing consolidation. Includes 25+ free models.

Pricing: Pay-per-token at provider price + 5.5% fee. No monthly minimum.

Best for: Rapid prototyping, research, small-to-medium workloads without infrastructure ops. At scale, the 5.5% fee compounds. No data residency guarantee.

License	Proprietary (managed)
GitHub	~7k (TS router)
Docs	openrouter.ai/docs

Cloudflare AI Gateway

What it is: Managed AI gateway built into Cloudflare's global edge network — zero infrastructure, geographic caching, DLP, BYOK (Bring Your Own Key), dynamic routing, and analytics.

Key features: Global edge caching (nearest PoP), DLP (Data Loss Prevention), rate limiting, guardrails, real-time analytics, provider routing and model fallbacks.

Best for: Teams already running traffic through Cloudflare who want caching, rate limiting, DLP, and analytics with zero ops. Less compelling if you only need simple model abstraction.

Kong AI Gateway

What it is: Enterprise API management platform extended to AI traffic — Kong's plugins + LLM-specific features (token-based rate limiting, cost estimation, AI routing, semantic caching).

Key features: Mature enterprise API management, Kubernetes-native, Kong AI Plugin for LLM routing + cost tracking, MCP support, RBAC, audit logs.

Best for: Organizations already using Kong as their API gateway who want a unified layer for both REST APIs and LLM traffic. Steep learning curve for non-Kong users.

Helicone

What it is: Developer-centric logging and observability proxy — the simplest gateway to add for immediate cost visibility and request logs.

Key features: Request logging, per-key analytics, cost dashboards, performance debugging, user tracking, prompt templates (Helicone Prompts).

Best for: Small-to-mid teams who want **immediate visibility into usage and costs** with minimal setup. Pairs well as an observability layer in front of LiteLLM routing.

TensorZero

What it is: Newest entrant — an open-source gateway with a focus on **structured output and function calling optimization**, plus a feedback loop for model improvement over time.

Key features: Structured output enforcement across providers, inference-time optimization, feedback-driven model routing, experiment framework.

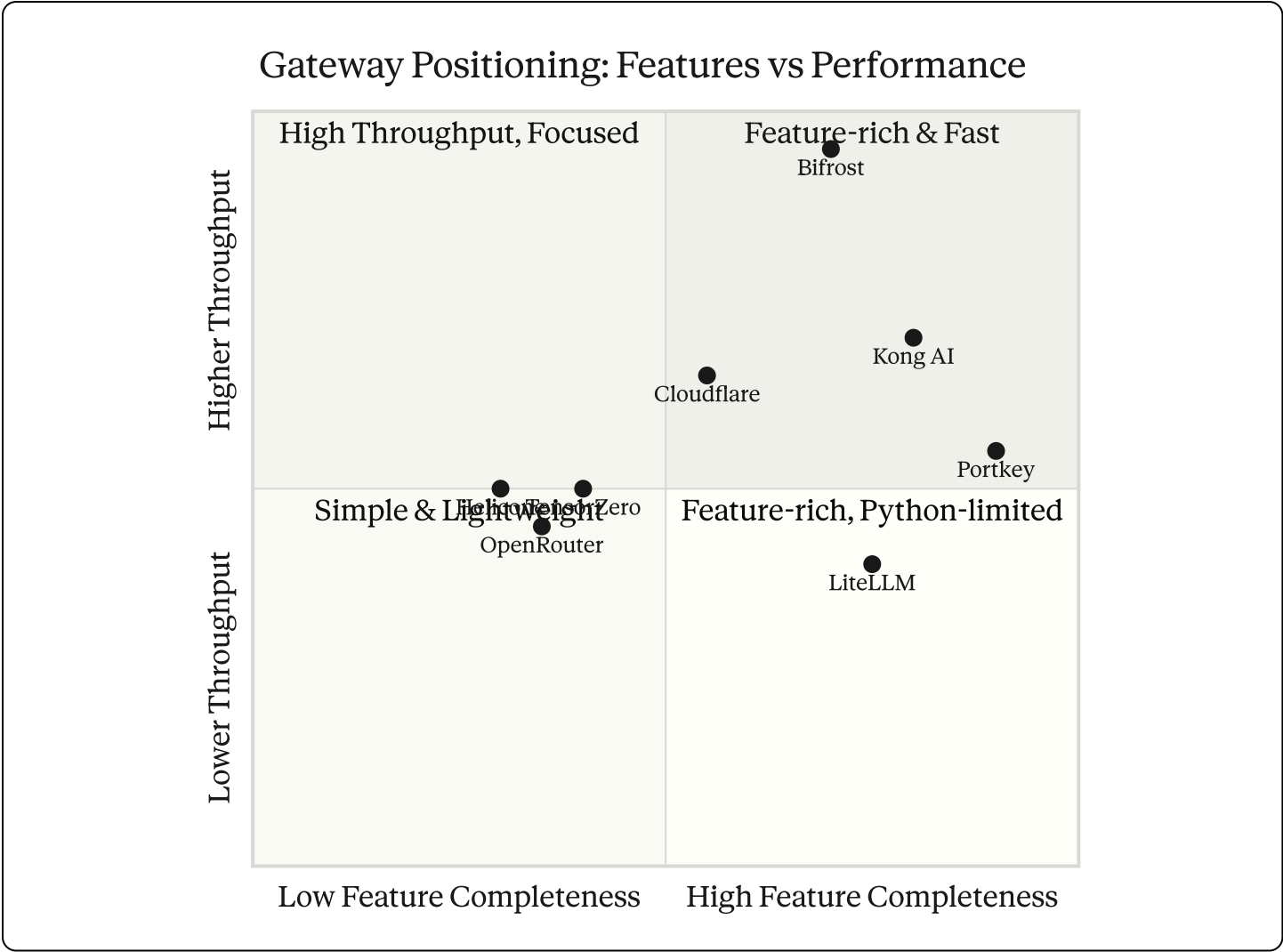
Best for: Teams building structured output pipelines who want to optimize which model handles each request based on observed quality.

Envoy AI Gateway

What it is: Extension of the CNCF Envoy proxy for LLM traffic — adds AI-specific features (token-based rate limiting, cost estimation, LLM routing) to an existing Envoy/Istio service mesh.

Best for: Teams with Envoy already in their Kubernetes stack (Istio users) who want to extend their existing proxy rather than add new infrastructure. Very early stage — limited provider support, no semantic caching.

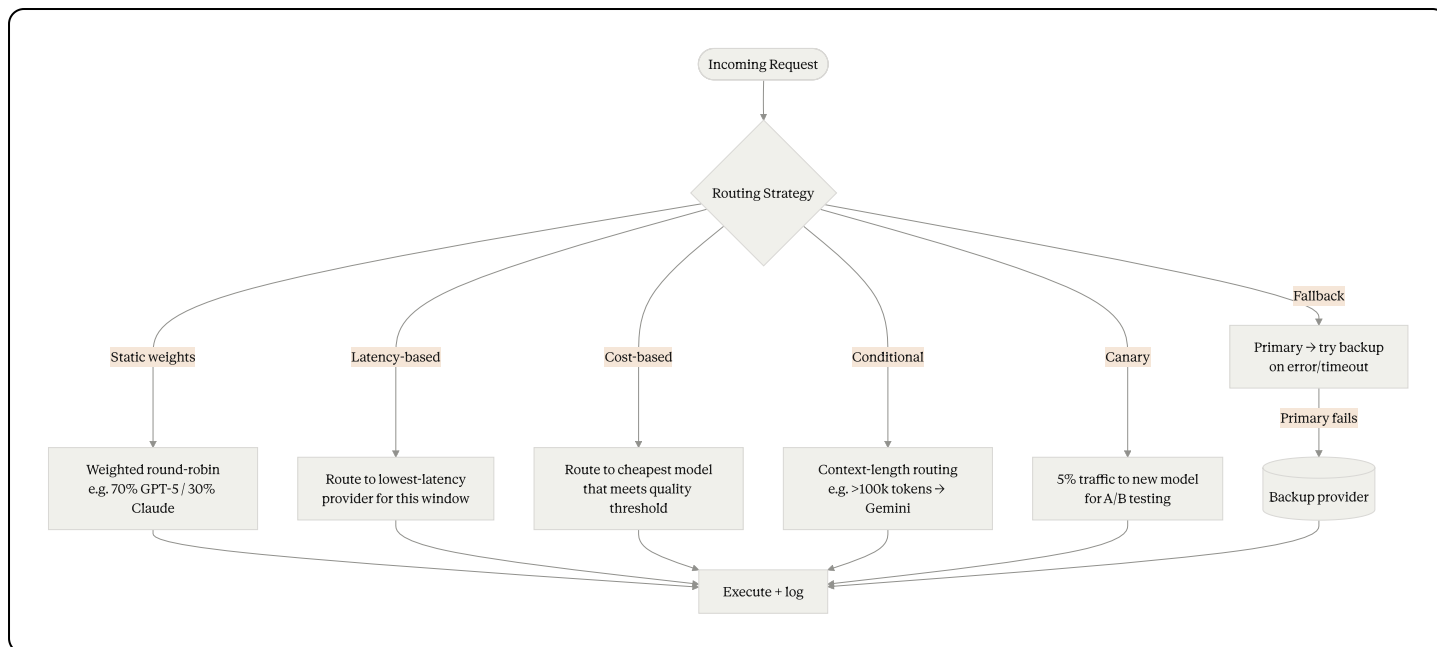
4. Feature Comparison Matrix



Feature	Portkey	Bifrost	LiteLLM	OpenRouter	Cloudflare	Kong
License	Apache 2.0	Apache 2.0	MIT	Proprietary	Proprietary	Apache 2.0
Provider count	1,600+	15+	100+	300+	30+	30+
Gateway overhead	~1ms	11μs	~8ms	<50ms (+network)	Edge-optimized	~2ms
Self-host	✓	✓	✓	✗	✗	✓
Semantic caching	✓ (best-in-class)	✓	✗ (exact only)	✗	✓ (edge)	Plugin
Virtual keys	✓	✓	✓	✗	✗	✓
RBAC / Teams	✓	✓	✓	✗	Limited	✓
Built-in guardrails	✓ 50+	Partial	Basic	✗	✓ DLP	Plugin
MCP Gateway	✓ (mature)	✓ (newer)	✓ (newer)	✗	✗	✓
Observability depth	✓✓ (deepest)	✓ (+ Maxim)	✓ (via integrations)	Basic	✓ analytics	✓
Prompt management	✓	✗	✗	✗	✗	✗
A/B testing / canary	✓	✓	✓	✗	✗	✓
Batch API	✓	✗	✗	✗	✗	✗
SOC 2 / HIPAA	✓	N/A (self-host)	N/A (self-host)	✓	✓	✓
Free tier	✓	✓ (OSS)	✓ (OSS)	✓ (free models)	✓	✓ (OSS)

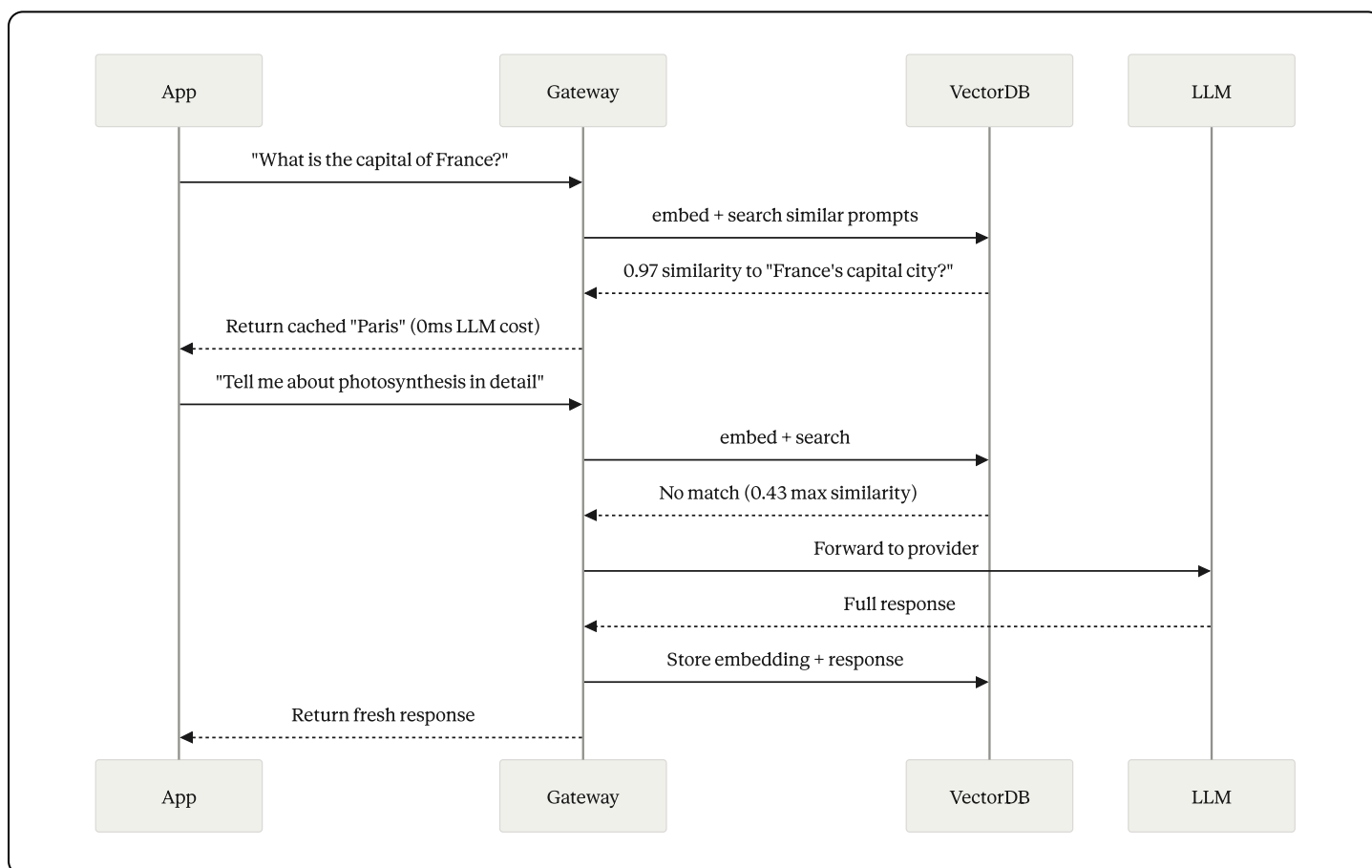
5. How Routing Works

Routing is the gateway's most important job in production. Here are the key strategies:



Semantic caching — how it works and why it matters

Semantic caching is one of the highest-ROI gateway features. Rather than exact-match caching (same prompt → same response), semantic caching embeds prompts and matches against a vector store of cached responses. If cosine similarity exceeds a threshold, the cached response is returned without an LLM call.



Portkey has the most sophisticated semantic caching (configurable similarity threshold, TTL, scope per virtual key). **Bifrost** also implements semantic caching via its vector store plugin. **LiteLLM** only supports exact-match Redis caching — a meaningful gap for workloads with semantically similar (but not identical) queries.

6. MCP Gateway — The New Frontier

Model Context Protocol (MCP) has fundamentally changed LLM gateways in 2026. MCP allows agents to call external tools (databases, APIs, file systems, code executors) — but without governance, any agent can call any tool with no audit trail.

An **MCP Gateway** adds the same control plane concepts (auth, RBAC, rate limiting, audit logs) to tool connections that LLM gateways add to model connections:

Portkey's MCP Gateway: Most mature — authentication, access control (revoke access instantly), full audit log of every tool call (who called what, parameters, response, latency), identity forwarding, MCP registry. Works with Claude Desktop, Cursor, VS Code.

Bifrost's MCP Gateway: In active development — centralize all MCP tool connections, governance, security, and auth. Centralized policy enforcement.

LiteLLM's MCP Gateway: Control tool access by team and key with granular permissions.

As Portkey CEO Rohit Agarwal said (March 2026): *"MCP has completely changed what it means to run AI in production. Six months ago, the conversation was about managing LLM traffic; now, enterprises are asking how they govern agents that can actually take actions inside their systems."*

7. Use Case → Gateway Mapping

Detailed scenarios

High-traffic customer-facing API (>1,000 RPS)

Primary concern: Gateway overhead cannot add meaningful latency to LLM calls; provider outages must be invisible to users.

Recommended: Bifrost — 11µs overhead, automatic failover, adaptive load balancing, no GIL ceiling. Pair with Maxim AI for observability or export OTel traces to Grafana/Datadog.

Configuration pattern:

```
yaml

providers:
  - name: primary
    provider: openai
    model: gpt-4o-mini
  - name: backup
    provider: anthropic
    model: claude-haiku-4-5
routing:
  strategy: latency_based
  fallback_on: [timeout, error_5xx]
```

Enterprise multi-team deployment

Primary concerns: Cost control per team, RBAC, audit logs for compliance, prompt management, MCP governance.

Recommended: Portkey — deepest observability, virtual keys with team budgets, RBAC, full MCP gateway, SOC 2 compliant. The cost tracking alone ("30 million policies a month, 25 GenAI use cases" — real Portkey customer quote) justifies adoption.

Configuration pattern:

```
python

from portkey_ai import Portkey

# Team A gets GPT-4o with $5k/month budget
client = Portkey(
    api_key="portkey_api_key",
    virtual_key="team_a_virtual_key", # Has $5k budget limit
    config={
        "strategy": {"mode": "fallback"},
        "targets": [
            {"virtual_key": "openai_key", "model": "gpt-4o"},
            {"virtual_key": "anthropic_key", "model": "claude-opus-4-6"}
        ]
    }
)
```

Research / prototype / maximum provider breadth

Primary concern: Try any model from any provider quickly; cost visibility; no infrastructure ops.

Recommended: **LiteLLM** for self-hosted maximum coverage, or **OpenRouter** for zero-infrastructure access. LiteLLM's 100+ providers means you never hit a "sorry, we don't support that provider" wall.

Migration path: Start with OpenRouter (instant access), move to LiteLLM proxy (more control) when team grows, upgrade to Portkey or Bifrost when traffic exceeds 500 RPS or governance matters.

RAG-heavy applications with repeated queries

Primary concern: Reduce LLM costs on frequently repeated or semantically similar queries.

Recommended: **Portkey** (best-in-class semantic caching) or **Bifrost** (semantic caching via vector store). LiteLLM only offers exact-match caching — insufficient for RAG where slightly rephrased questions should hit the cache.

Expected savings: 30–70% cost reduction on workloads with high semantic similarity (embedding lookups, document Q&A with similar questions).

Agent / agentic workflows with MCP tools

Primary concern: Govern which agents can use which tools; audit every tool call; prevent runaway agent actions.

Recommended: **Portkey MCP Gateway** (most mature governance) + **Bifrost** (if performance is also critical). Both provide centralized auth, RBAC, and audit logs for MCP tool access.

Regulated industries (healthcare, finance, legal)

Primary concerns: Data residency (prompts must not leave your infrastructure), audit logs for compliance, HIPAA/GDPR, SOC 2.

Recommended: **Self-hosted Portkey** or **Bifrost** — both run fully in your infrastructure, nothing leaves except the actual LLM API call (which is encrypted in transit). LiteLLM also works. Avoid managed-cloud gateways if strict data residency is required.

Add: **LLM Guard** for offline PII scanning before prompts leave your gateway to the LLM provider.

Small team, quick start

Recommended path:

1. Start with **OpenRouter** — instant access, no setup, test models
 2. Add **Helicone** as an observability proxy when you need cost visibility
 3. Migrate to **LiteLLM** when you need self-hosting or custom routing
 4. Upgrade to **Portkey** or **Bifrost** when you need semantic caching, guardrails, or team governance
-

8. How to Evaluate a Gateway (Decision Framework)

Before choosing, answer these questions:

The 10-question checklist

Question	Implication
Do you need >1,000 RPS from the gateway itself?	Yes → Bifrost or Kong; LiteLLM GIL may bottleneck
Do prompts need to stay within your VPC?	Yes → self-hosted only (Bifrost, LiteLLM, Portkey self-host)
Do you need 100+ provider options?	Yes → LiteLLM; Bifrost and Portkey cover fewer
Do you need semantic (fuzzy) caching?	Yes → Portkey or Bifrost; LiteLLM = exact-match only
Do you need MCP tool governance?	Yes → Portkey (most mature) or Bifrost
Do you need built-in guardrails at gateway?	Yes → Portkey (50+ built-in); others need external
Do you need prompt version management?	Yes → Portkey (others don't have this)
Do you need multi-team RBAC + budgets?	Yes → Portkey or LiteLLM; Bifrost has governance plugin
Is infrastructure ops a burden to minimize?	Yes → OpenRouter or Portkey managed
Do you need SOC 2 / HIPAA with a BAA?	Yes → Portkey managed (BAA), Cloudflare, Kong Enterprise

9. Architecture Patterns

Pattern 1: Simple routing layer

Best for: Prototyping, small teams, single-purpose apps.

Pattern 2: Full observability + routing

Best for: Production apps at moderate scale needing cost visibility.

Pattern 3: Enterprise control plane

Best for: Enterprise with multiple teams, compliance requirements, agent governance.

Pattern 4: High-throughput, latency-critical

Best for: Customer-facing features at 5,000+ RPS.

10. Cost Architecture: Making the Business Case

Where gateways save money

Mechanism	Typical savings	Notes
Semantic caching	30-70% on repeated/similar queries	Highest ROI for RAG, support bots, coding assistants
Intelligent routing	20-60% cost reduction	Route to cheapest model that meets quality threshold
Budget enforcement	Prevent overspend	Catch a runaway agent before it spends \$50k
Batch routing	40-60% on async tasks	Provider batch APIs are much cheaper than real-time
Prompt optimization	Variable	Identify verbose prompts; shorten via prompt management

Real numbers (from public Portkey customer quotes, May 2026)

- "Saved us thousands of dollars by caching tests that would otherwise run repeatedly" (GitHub workflows)
- "30 million policies a month, managing over 25 GenAI use cases" (enterprise)
- "\$180M in annualized LLM spend governed" through Portkey across 24,000+ organizations

Gateway ROI formula

Monthly savings = (cache_hit_rate × monthly_tokens × cost_per_token)
+ (routing_savings_% × monthly_spend)
- (gateway_infra_cost)

Break-even typically at: ~\$500/month in LLM spend (self-hosted OSS gateway)

11. Quick Reference: GitHub Repos & Docs

Gateway	GitHub	Docs	Stars
Portkey	github.com/Portkey-AI/gateway	portkey.ai/docs	~7k+
Bifrost	github.com/maximhq/bifrost	docs.getmaxim.ai/bifrost	~2k+
LiteLLM	github.com/BerriAI/litellm	docs.litellm.ai	~20k+
OpenRouter	openrouter.ai	openrouter.ai/docs	~7k (TS)
Cloudflare AI GW	(managed service)	developers.cloudflare.com/ai-gateway	N/A
Kong AI GW	github.com/Kong/kong + AI plugins	docs.konghq.com	~40k+ (Kong)
Helicone	github.com/Helicone/helicone	docs.helicone.ai	~1.5k
TensorZero	github.com/tensorzero/tensorzero	tensorzero.com/docs	~5k+
Envoy AI GW	github.com/envoyproxy/ai-gateway	envoyproxy.io	~25k (Envoy)

Conclusion

LLM gateways have graduated from "nice to have" to **critical AI infrastructure** in 2026. The choice comes down to three primary dimensions:

Performance: If you're serving >1,000 RPS from a customer-facing feature, **Bifrost's Go-based 11µs overhead** is the answer. Python-based gateways (LiteLLM) hit the GIL ceiling at moderate scale.

Feature completeness: If you need observability depth, semantic caching, built-in guardrails, prompt management, MCP governance, and enterprise RBAC in one platform, **Portkey** is the clear leader — and it's now fully open-source.

Provider breadth: If you need to access 100+ providers with maximum ecosystem compatibility and you're comfortable managing Python infrastructure, **LiteLLM** remains the most comprehensive OSS option.

The pragmatic default for most production teams: Start with **Portkey** (cloud-managed or self-hosted) for its combination of feature completeness, observability, and MCP governance. Migrate specific high-traffic routes to **Bifrost** if latency becomes the bottleneck. Use **LiteLLM** if you hit a provider that Portkey doesn't cover.

The MCP Gateway layer is the next major battleground — as agents proliferate in 2026 and beyond, governing tool access will become as critical as governing model access. The gateways that get MCP governance right will define the enterprise AI infrastructure stack for the next five years.